PART III · CHAPTER 15

Flight and Swim Locomotion

The 6-DOF dynamics layer that turns guidance into a body that moves believably.

Ultra-realistic 6-DOF locomotion for flying/swimming pawns (drones, birds, fish), layered on top of LCMNav3D navigation. The nav system provides **guidance** (where to go); this layer provides physically-plausible **dynamics** (how the body moves).

Concept: Guidance → Control → Dynamics

- **Nav (unchanged):** SVO path / flow field → pure-pursuit + CBS/ORCA avoidance → a *desired velocity*.
- `ULcmFlightMovementComponent` **(new):** consumes that desired velocity and runs a 6-DOF rigid-body flight/hydro model (thrust, lift/drag, gravity/buoyancy, attitude PID) per archetype, producing the pawn transform **and** an animation-drive struct.

Quick start

1. Add a **Lcm Flight Movement Component** to a pawn whose root is a collision primitive.
2. Set: `bEnabled = true`, `LocomotionModel = FlightDynamics`, `Archetype` (Multirotor / Bird / Fish / Generic6DOF), and tune `FlightConfig`.
3. Drive it one of two ways:
   - **Nav-driven (recommended):** on the pawn's **BT/StateTree FlyTo** task, tick `Use Flight Movement Component`. The task feeds guidance automatically.
   - **Manual:** call `RequestVelocity(WorldDesiredVelocity)` each tick from your own code/Blueprint.

Off by default: with `bEnabled=false` (or the task flag off) nothing changes; motion is the legacy kinematic FlyTo.

Archetypes (key `FlightConfig` params)

Archetype	Model	Notable params
Generic6DOF	Thrust-along-`ThrustAxis` body, gravity-on, attitude PID	`Mass`, `MaxThrust`, `InertiaTensor`, `VelocityP`, `AttitudeP/D`, `LinearDragCoeff`, `GravityScale`
Multirotor	Tilt-to-translate, altitude hold, spool lag, wind/gust, hover jitter	`RotorSpinUpRate`, `WindVelocity`, `GustStrength/Frequency`, `HoverJitterStrength`, `bYawTracksVelocity`, `RotorMaxRPM`
Bird	Forward pulsed wingbeat thrust, airspeed² lift + AoA control, flap↔glide, coordinated banks, flare	`FlapFrequency`, `FlapThrust`, `LiftCoeff`, `MaxLiftFactor`, `BankTurnGain`, `MaxBankAngleDeg`, `FlareSpeed`, `FlareDragScale`
Fish	Neutral buoyancy, tail-beat undulatory thrust + sway, anisotropic hydro drag, current	`bNeutralBuoyancy`, `TailBeatFrequency`, `TailBeatFreqGain`, `TailWiggleStrength`, `LateralDragMult`, `WaterCurrent`

Shared: `MaxSpeed`, `MaxAcceleration`, `MaxTurnRateDeg`, `SubSteps` (integration sub-steps; higher = stiffer + smoother), `BankingAmount` / `MaxBankingAngle` (kinematic-mode cosmetic roll).

Notes:

- A fast forward-flyer (bird/fish) can't pinpoint-stop; the nav layer's **arrival slowdown** makes it flare to a perch. Without arrival-aware guidance a fast body orbits its goal (expected).
- Realistic dynamics *lag* guidance by design; raise `MaxAcceleration` / `AttitudeP` for snappier, lower for heavier/driftier.

Animation contract: `FLcmFlightState`

Every tick the component produces `FLcmFlightState` (archetype-agnostic where possible):

Field	Meaning
`Speed01`	Normalised airspeed 0..1 (|velocity| / MaxSpeed), used as a blendspace axis
`VerticalSpeed`	cm/s, climb + / dive −; climb/dive pose blend
`BankAngleDeg` / `PitchDeg`	Body roll / pitch, for additive lean/pitch
`YawRateDeg`	Turn rate, for lean/tilt into turns
`GaitPhase`	0..1 cyclic driver: rotor spin / wingbeat / tail-beat
`GaitAmplitude01`	Throttle (drone) / flap-glide blend (bird) / undulation vigour (fish)
`RotorRPM`	Multirotor rotor RPM
`Throttle01` / `Airspeed`	Commanded effort / raw speed

Consume it two ways:

- **Pull:** `GetFlightState()` (BlueprintPure) on the component, e.g. from an AnimBP's Update.
- **Push:** implement `ILcmFlightAnimInterface::ReceiveFlightState` on your AnimBP (or pawn). The component calls it every tick (`bBroadcastAnimState` , default on). Cache the struct and drive:
  - `GaitPhase` → rotor/wing/tail cyclic pose (Sine/curve on the phase)
  - `Speed01` + `VerticalSpeed` → locomotion blendspace
  - `BankAngleDeg` / `PitchDeg` → additive lean/pitch
  - `GaitAmplitude01` → flap-vs-glide (bird) or effort (fish/drone) blend weight
  - `RotorRPM` → prop-spin material/WPO param (Multirotor)

Mass crowds & scale (P7-P8)

The **LCM Nav3D Agent** trait (NavMode = PathFollowing) carries the same flight options (`LocomotionModel` , `FlightArchetype` , `FlightConfig`): set them and a whole crowd flies with the identical 6-DOF physics as a pawn, still pathing + avoiding + arriving. Default `Kinematic` → byte-identical to before.

Dynamics LOD (the scale knob). Full 6-DOF per agent is expensive; set `FlightDynamicsLODDistance` (cm) so only agents within that distance of the viewer run full dynamics; everyone else uses the cheap kinematic integrator (face-velocity, no roll). So a 10K crowd keeps only the on-screen agents on the expensive path. `0` = always full. Global kill switch: `r.LcmNav.Mass.FlightLOD` (1 default / 0 = full everywhere). LOD switching does not pop: a far agent's integrator state re-syncs from its live transform when it returns to range.

Deferred (documented) scale extensions:

- **GPU 6-DOF compute** for very large crowds is a designed extension, intentionally not shipped here: Mass GPU paths on this engine version are RDG/render-thread crash-prone (see the plugin's Mass notes), and

CPU dynamics-LOD already bounds cost to the visible set. Revisit with engine uplift.

- **Shared flight config**: `FlightConfig` is currently per-entity; moving it to a const-shared fragment per archetype would cut crowd memory. Deferred (shared-fragment archetype plumbing).

Determinism

The dynamics are deterministic: fixed integration sub-steps, no RNG or wall-clock; gust/jitter are seeded per-agent sinusoidal noise. Same input sequence → identical trajectory (headless-verified by `-run=LcmFlightDynamicsTest`).

Chapter 15 of the LCM Nav3D User Manual.
The complete manual is `LCM_Nav3D_Manual.pdf`, in this folder alongside every other document in the set. Start from `README.pdf`.